

Implementation of Syntax analyzer using the tool YACC.

YACC (Yet Another Compiler Compiler) is a parser generator that converts a context-free grammar into a syntax analyzer (parser). It works in conjunction with LEX (a lexical analyzer).

Structure of a YACC Program

A YACC program has three sections:

1. **Definition Section:**

- Includes C declarations and `%token` declarations.
- Used to define token names and precedence.

2. **Grammar Rules Section:**

- Specifies parsing rules using BNF-like syntax.
- Each rule can include associated actions written in C.

3. **User Code Section:**

- Contains `main()` function and error handlers.

Compiling and Running YACC + LEX

```
lex file.l
yacc -d file.y
gcc lex.yy.c y.tab.c -ll
./a.out
```

Program 1: Recognize valid arithmetic expressions using operators and parentheses.

Algorithm

1. Define tokens for numbers and operators.
2. Use precedence declarations to resolve ambiguity.
3. Define grammar rules for:
 - Binary operations: `+`, `-`, `*`, `/`
 - Parentheses

- Single number
4. If parsing is successful, print “Valid expression.”
 5. If parsing fails, handle with `yyerror()`.

YACC Code

```
%{
#include <stdio.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex(void);
%}

%token NUMBER
%left '+' '-'
%left '*' '/'
%nonassoc '(' ')'

%%

expression:
    expression '+' expression
  | expression '-' expression
  | expression '*' expression
  | expression '/' expression
  | '(' expression ')'
  | NUMBER
  ;

%%

void yyerror(const char *s) {
    printf("Invalid expression: %s\n", s);
}

int main() {
    printf("Enter an arithmetic expression:\n");
    if (yyparse() == 0) {
        printf("Valid expression\n");
    }
    return 0;
}
```

Corresponding LEX Code

```
%{
#include "y.tab.h"
%}

%%

[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t\n]     { }
"+"         { return '+'; }
"-"         { return '-'; }
"*"         { return '*'; }
"/"         { return '/'; }
"("         { return '('; }
")"         { return ')'; }
.           { return yytext[0]; }

%%

int yywrap() { return 1; }
```

Sample Output

```
Enter an arithmetic expression:
(3+5)*2
Valid expression
```

Program 2: Implement Calculator using YACC and LEX

Algorithm

1. Define token NUMBER for digits.
2. Set precedence for operators +, -, *, /.
3. Grammar rules build expressions recursively.
4. Actions compute the result using

=

1 + 3, etc. Store final result in a global variable and print it.

YACC Code (Calculator)

```
%{
#include <stdio.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex(void);
int result;
%}

%token NUMBER
%left '+' '-'
%left '*' '/'

%%

expression:
    expression '+' expression    { $$ = $1 + $3; result = $$; }
  | expression '-' expression    { $$ = $1 - $3; result = $$; }
  | expression '*' expression    { $$ = $1 * $3; result = $$; }
  | expression '/' expression    { $$ = $1 / $3; result = $$; }
  | NUMBER                       { $$ = $1; result = $$; }
  ;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main() {
    printf("Enter an arithmetic expression:\n");
    if (yyparse() == 0) {
        printf("Result: %d\n", result);
    }
    return 0;
}
```

LEX Code for Calculator

```
%{
#include "y.tab.h"
%}

%%
```

```

[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t\n]     { }
"+"        { return '+'; }
"-"        { return '-'; }
"*"        { return '*'; }
"/"        { return '/'; }
.          { return yytext[0]; }

%%

int yywrap() { return 1; }

```

Sample Output

5. Enter an arithmetic expression:
 8+4*2
 Result: 16